

基于多目标优化的城市应急物资调度研究

摘 要

针对突发事件中需求不确定、道路通行能力动态变化和多配送中心协同困难的问题，本文构建兼顾响应时间、运输成本与服务公平性的多目标应急物资调度模型。

首先，对需求、库存和路网数据进行一致性校验与异常值处理，并用区间参数描述需求波动。其次，将车辆容量、物资守恒、时间窗和道路容量写入约束，以加权切比雪夫方法获得帕累托折中解。再次，通过滚动时域策略更新需求预测和路径决策。算例结果表明，该方法能够在满足关键需求点最低保障率的同时控制总成本；敏感性分析用于识别需求波动和道路容量对方案稳定性的影响。

最后，从数据误差、参数扰动和极端情景三个层面检验模型，并讨论模型的适用边界。本文方法可为同类多源供给、动态需求的资源配置问题提供可复用的决策框架。

关键词：应急调度；多目标优化；鲁棒性；敏感性分析

1 问题重述

用自己的语言提炼任务目标、已知条件和待求量，不逐句复制赛题。正文从摘要页后的下一页开始，全文不设置目录。

2 问题分析

将每个子问题拆解为数据、决策变量、约束、目标和验证方式，并说明各子问题之间的数据流与逻辑关系。

2.1 总体思路

本文先完成数据质量检查，再建立静态基准模型，随后加入不确定性与动态更新机制，最后通过基准对比、敏感性分析和极端情景测试检验结论。

3 模型假设

- 1) 赛题给出的原始数据真实有效；对缺失值和异常值的处理方法在正文中明确说明。
- 2) 单个决策周期内车辆平均速度保持不变；跨周期变化由滚动更新刻画。
- 3) 未显式建模的次要因素对核心结论不产生方向性改变。

4 符号说明

表格采用三线表。较长说明列使用自动换行的居中列类型 Y；数值列可使用自动换行的右对齐列类型 R。

表 1 主要符号与定义

符号	含义	单位
x_{ijk}	车辆 k 是否由节点 i 行驶 至节点 j	—
q_{ij}	配送中心 i 向需求点 j 配 送的物资量	kg
t_j	需求点 j 的实际到达时间	min

5 模型建立与求解

5.1 数据预处理

记录数据来源、清洗规则、缺失值比例和异常值判据。所有自主查阅的数据及处理脚本应同时收入支撑材料。

5.2 多目标模型

令运输成本、响应时间和服务公平性损失分别为 f_1, f_2, f_3 ，标准化后的综合目标为

$$\min F = \max_{r \in \{1,2,3\}} \left\{ w_r \frac{f_r - f_r^{\min}}{f_r^{\max} - f_r^{\min}} \right\}, \quad (1)$$

其中 $w_r \geq 0$ 且 $\sum_r w_r = 1$ 。物资守恒约束可写为

$$\sum_i q_{ij} \geq d_j, \quad \sum_j q_{ij} \leq s_i. \quad (2)$$

5.3 求解流程

报告算法参数、终止条件、随机种子、运行环境与重复次数。结果表必须给出量纲、统计口径和必要的不确定性指标。

表 2 不同方案的示例结果

方案	是否满足全部 约束	总成本/万元	平均响应时 间/min
基准方案	是	128.40	47.6
鲁棒方案	是	134.10	41.2
动态方案	是	131.75	38.9

6 模型检验与敏感性分析

围绕输入误差、关键参数和极端情景分别开展检验，并说明结论在何种扰动范围内保持稳定。不能只报告“效果较好”，应给出可复核的量化证据。

7 模型评价与推广

总结模型的解释性、计算效率、数据依赖和适用边界。推广时明确哪些结构可复用，哪些参数必须重新估计。

AI 工具使用声明

本参赛队在竞赛过程中未使用任何 AI 工具。

参考文献

- [1] 姜启源, 谢金星, 叶俊. 数学模型[M]. 3 版. 北京: 高等教育出版社, 2003.
- [2] 张三, 李四. 不确定条件下的多目标应急调度方法[J]. 系统工程理论与实践, 2026, 46 (1): 1-12.

A 支撑材料文件列表

支撑材料应单独压缩为 RAR 或 ZIP, 且不超过 20 MB。示例清单如下; 提交前必须与实际压缩包逐项一致。

序号	文件路径	内容说明
1	code/example.py	可运行的示例程序
2	data/processed.csv	自主获取并清洗后的数据 (示例, 按实际替换)
3	AI 工具使用详情.pdf	仅在使用 AI 工具时提供

B 完整源程序

建模所用的全部源程序必须完整、可运行, 并与论文结果一致。此处演示从文件直接载入代码, 避免论文附录与支撑材料中的代码版本漂移。

Listing 1 示例程序

```
1 """Minimal reproducible example for the appendix and supporting materials."""
2
3 from __future__ import annotations
4 def weighted_objective(values: list[float], weights: list[float]) -> float:
5     """Return a weighted Chebyshev objective for normalized criteria."""
6     if len(values) != len(weights):
7         raise ValueError("values and weights must have the same length")
8     if abs(sum(weights) - 1.0) > 1e-9:
9         raise ValueError("weights must sum to one")
10    return max(value * weight for value, weight in zip(values, weights))
11 if __name__ == "__main__":
12     sample = weighted_objective([0.42, 0.36, 0.51], [0.4, 0.35, 0.25])
13     print(sample)
```